UN MANUALE TECNICO DI BM

Il linguaggio bml

Il sottoinsieme tipato di Lua per il Pi Zero W: sintassi, VM a stack, bmvm del kernel e codice nativo ARMv6 A32 con equivalenti C e ASM.

Indice

Perché un linguaggio di bm 3

Una sorgente, quattro bersagli 3

Sintassi e regole di tipo 4

Tipi e letterali 4

Istruzioni e operatori 5

Come funziona il compilatore 6

Tappa 1 e Tappa 2: AST e bytecode a stack 6

Tappa 3 e Tappa 4: `bmvm` del kernel e ARMv6 A32 6

Esempi pratici di codice: BML, C e ASM ARMv6 7

Esempio 1: Somma in un ciclo (`sum`) 7

Esempio 2: Limitazione di un valore (`clamp`) 7

Esempio 3: Massimo comun divisore di Euclide (`gcd`) 8

Esempio 4: Doppia ricorsione (`fib`) 9

Esempio 5: Struct, array e `f32` VFPv2 (`step_vec2f`) 10

Esempio 6: Ottimizzazione di una cartuccia (Dark Night) 11

Prova passo per passo 13

Punti aperti 14

LEGGIMI

Che cos'è. Come scrivere funzioni calde nel linguaggio tipato di bm (`tools/bml/bml.py`), eseguire la stessa sorgente in Lua standard e ispezionare il bytecode a stack, il codice a registri `bmvm` del kernel, l'equivalente C e le istruzioni macchina native ARMv6 A32 a 32 bit per il Raspberry Pi Zero W.

Come leggerlo. Ogni regola, opcode e parola macchina viene da `tools/bml/bml.py` , `scripts/bmt.py` , `src/bm/bmvm.h` e `docs/design/bml-manual.md` . Il capitolo 4 mostra quattro funzioni complete con sorgente BML, equivalente C e assembly ARMv6 disassemblato fianco a fianco.

Che cosa manca. Ogni tappa aperta della roadmap in `docs/progress/typed.md` è segnata con un'etichetta DA COMPLETARE evidenziata ed elencata nell'ultimo capitolo.

Perché un linguaggio di bm

Un gioco sul Raspberry Pi Zero W 1.1 gira su un singolo core ARM1176JZF-S a 1000 MHz con un budget per frame di 16,7 ms a 60 fps (`docs/PERFORMANCE.md`). Quando il profiler Lua (`profile(true)`) mostra una funzione calda chiamata migliaia di volte per frame, serve la velocità del codice nativo senza mantenere due sorgenti diverse.

Una sorgente, quattro bersagli

Un file `.b` usa la sintassi di Lua con annotazioni di tipo (`: i32` , `: bool`) sui parametri e sul valore di ritorno. Da un solo file `.b` , `tools/bml/bml.py` produce:

Bersaglio	Opzione / API	Dove gira	Scopo
Lua 5.4 standard	`--lua`	`luahost` e la VM Lua della console	Oracolo di riferimento: rimuove `: <tipo>` e `struct ... end`
VM a stack (Tappa 2)	`--run` / `--dump`	Python sul PC (`StackVM`)	Verifica rapida e ispezione del bytecode
`bmvm` del kernel (Tappa 3)	`--bmvm`	`src/bm/bmvm.c` (`vm_load` , `vm_run`)	Blob sicuro per macchina a registri dentro la cartuccia
ARMv6 A32 nativo (Tappe 4–8)	`--armv6` / `--native`	`src/bm/bmnative.c` (`native_load` , `native_run`)	Parole macchina A32 + VFPv2 a 32 bit in RAM eseguibile (AAPCS)
Compilatore su console (Tappa 9)	`native_compile(src)`	`src/bm/bmlc.c` (`bml_compile_native`)	Compila testo `.b` direttamente in codice macchina ARMv6 `BMN1` sul Pi

NOTA

`scripts/bmt.py` traduce la stessa sintassi Lua tipata in C (`--c`) per le funzioni collegate nel kernel (`src/bm/api_collmap.c`). `tools/bml/bml.py` è il compilatore autonomo che genera direttamente bytecode e parole macchina ARMv6 senza un compilatore C.

Sintassi e regole di tipo

Il front end (`Lexer` , `Parser` , `AST` , `TypeChecker` in `tools/bml/bml.py`) controlla ogni istruzione prima di emettere codice. Ogni errore ferma la compilazione e stampa `line <N>: <reason>` .

Tipi e letterali

Tipo	Significato	Letterali / Sintassi	Rappresentazione ARMv6
`i32`	Intero con segno a 32 bit (wrap in complemento a 2)	`0` , `42` , `0x1234` , `0xFFFFFFFF` , `-2147483648` , `2147483647`	Registro generale a 32 bit (`r0` .. `r10`)
`bool`	Valore di verità stretto	`true` , `false`	`1` (`true`) o `0` (`false`) in un registro a 32 bit
`f32`	Float IEEE-754 a singola precisione	`0.0` , `1.5` , `-0.25`	Slot GP a 32 bit + coprocessore 10 VFPv2 `s0` .. `s1`
`string`	Stringa immutabile e costanti nel segmento `.rodata`	`"testo"` , `'testo'`	Puntatore a 32 bit nel segmento `.rodata` (PIC PC-relative, zero rilocazioni)
`i32[]` , `bool[]` , `f32[]`	Array tipati con indice base 1	`a[i]` , `a[i] = v`	Puntatore base in registro a 32 bit (`ldr` / `str` a `base + (i - 1) * 4`)
`struct Name ... end`	Record a layout fisso	`s.field` , `s.field = v`	Puntatore a parole contigue di 32 bit (`offset = field_idx * 4`)
`const NAME: tipo = val`	Costante a costo zero valutata a compile-time	`const MAX: i32 = 16` , `const S: string = "ok"`	Sostituita con valore immediato o offset `.rodata` nel codice macchina
`enum Name ... end`	Enumerazione di interi sequenziali	`enum State IDLE = 0, RUN, JUMP end`	Valutata a compile-time come `i32` (`State.RUN`)
`function Struct:metodo(...)`	Metodo associato a una struct	`function Vec2f:len_sq(): f32 ... end`	Primo parametro implicito `self` ; dispatch statico diretto (`v:len_sq()`)

- Le condizioni in `if` , `elseif` , `while` , `and` , `or` e `not` devono avere tipo `bool` . Un `i32` non è mai usato implicitamente come booleano.
- La divisione intera usa `//` e il modulo usa `%` (arrotondati verso $-\infty$). La divisione float usa `/` su operandi `f32` .
- Funzioni matematiche e di conversione integrate: `abs(x)` , `min(a, b)` , `max(a, b)` (per `i32` o `f32`), `sqrt(x: f32): f32` , `f32(x: i32): f32` e `floor(x: f32): i32` .

- Funzioni del kernel (Tappa 7+): `cls(col)`, `pix(x, y, col)`, `rectfill(x, y, w, h, col)`, `spr(id, x, y)`, `btn(b): bool`, `btnp(b): bool`, `line(x0, y0, x1, y1, col)`, `circ(x, y, r, col)`, `circfill(x, y, r, col)`, `rect(x, y, w, h, col)`, `sfx(id)`, `music(id)`, `time(): f32` e `print(msg, ...): i32` (fino a 5 argomenti; primo argomento `string`).
- Primitive di memoria bare-metal e MMIO (Tappa 5): `peek(addr: i32): i32` (`ldrb` byte a 8 bit), `poke(addr: i32, val: i32)` (`strb`), `peek2(addr: i32): i32` (`ldrh` halfword a 16 bit per audio e RGB565), `poke2(addr: i32, val: i32)` (`strh`), `peek4(addr: i32): i32` (`ldr` parola a 32 bit), `poke4(addr: i32, val: i32)` (`str`), `memcpy(dst: i32, src: i32, bytes: i32)` e `memset(dst: i32, val: i32, bytes: i32)` (routine macchina native ad alta efficienza per framebuffer, asset e DMA).

Istruzioni e operatori

Esempio	Regola		
`function add(a: i32, b: i32): i32 ... end`	Fino a 16 parametri; supporta ricorsione e chiamate a funzioni definite sopra		
`local s = 0` o `local s: i32 = 0`	Inferenza di tipo automatica (`local x = expr`) per scalari, struct e array o tipo esplicito; visibile fino alla fine del blocco		
`s = s + i`	La variabile deve essere dichiarata e avere lo stesso tipo		
`x += 1`, `s.fld *= 2`, `a[i] <<= 4`	Supporta `+=`, `-=`, `*=`, `&=`, `|=`, `<<=`, `>>=`; desugaring per Lua e nativo		
`local p: Struct = Struct(addr)`, `local a: i32[] = addr`	Accesso strutturato a zero-overhead su memoria MMIO/framebuffer/DMA		
`switch s do case 1 then ... default ... end`	Salto a caso senza fall-through; desugaring pulito per Lua e C nativo		
`if a < b then ... elseif a == b then ... else ... end`	Ogni condizione deve essere `bool`		
`while b ~= 0 do ... end`	La condizione deve essere `bool`; `break` esce dal ciclo più interno		
`for i = 1, n do ... end` o `for i = n, 1, -2 do ... end`	Estremi e passo opzionale sono `i32`		
`+`, `-`, `*`, `//`, `%`, `&`, `|`, `~`, `<<`, `>>`, `<`, `<=`, `>`, `>=`, `==`, `~=`, `and`, `or`, `not`	Bitwise (`&`, `|`, `~`, `<<`, `>>`) e precedenze		

Come funziona il compilatore

Tappa 1 e Tappa 2: AST e bytecode a stack

1. `tokenize()` divide il testo sorgente in record `Token(kind, text, line, start, end)` .
2. `Parser` costruisce l'albero sintattico `Program` (`FuncDecl` , `Stmt` , `Expr`) e annota gli intervalli di byte di `: i32` e `: bool` affinché `to_lua()` possa rimuoverli.
3. `TypeChecker` verifica gli ambiti delle variabili, i tipi degli operatori, le chiamate di funzione e controlla che ogni percorso di una funzione con tipo di ritorno termini con `return` .
4. `BytecodeCompiler` emette le istruzioni a stack (`PUSH_CONST` , `LOAD` , `STORE` , `ADD` , `SUB` , `MUL` , `IDIV` , `IMOD` , `BAND` , `BOR` , `BXOR` , `BNOT` , `SHL` , `SHR` , `LE` , `JZ` , `JF_KEEP` , `JT_KEEP` , `CALL` , `RET`).

Tappa 3 e Tappa 4: `bmvm` del kernel e ARMv6 A32

1. `compute_stack_depths()` calcola l'altezza esatta dello stack di valutazione prima di ogni istruzione.
2. `BMVMCompiler` mappa i parametri in `R[0..nparams-1]` , le costanti in `R[nparams..nparams+nconst-1]` , i locali in `R[nparams+nconst..]` e gli slot dello stack `S[k]` sopra i locali, generando un blob `BMV1` per `src/bm/bmvm.c` .
3. `ARMv6Compiler` emette parole macchina ARMv6 A32 a 32 bit secondo l'ABI AAPCS (`r0` .. `r3` argomenti, `r0` risultato, registri callee-saved `r4` .. `r10` per i primi 7 slot locali/stack e slot `[fp, #-off]` sullo stack per le variabili eccedenti).
4. `compile_native_blob()` (`--native`) impacchetta il modulo `BMN1` per `src/bm/bmnative.c` (`native_load` , `native_fn` , `native_run`): `bmnative_load` verifica le istruzioni A32, le copia in RAM eseguibile allineata a 32 byte, pulisce la D-cache (`dcache_clean_range`), invalida la I-cache (`mcr p15, 0, a, c7, c5, 1`) e il branch target cache (`c7, c5, 6`), ed esegue `DSB` + `ISB` .

Esempi pratici di codice: BML, C e ASM ARMv6

Esempio 1: Somma in un ciclo (`sum`)

Il ciclo canonico di `tests/bml/sum.b` somma gli interi da `1` a `n` .

```
-- 1. Sorgente BML (tests/bml/sum.b)
function sum(n: i32): i32
  local s = 0
  for i = 1, n do s = s + i end
  return s
end
```

```
/* 2. Equivalent C (scripts/bmt.py --c) */
int32_t bmt_sum(int32_t n)
{
    int32_t s = 0;
    {
        int32_t i__end = n;
        for (int32_t i = 1; i <= i__end; i++) {
            s = (s + i);
        }
    }
    return s;
}
```

```
; 3. ARMv6 A32 leaf machine code (9 words = 36 bytes, r0 = n, r1 = s, r2 = i)
  0x0000: E3A01000    mov   r1, #0          ; s = 0
  0x0004: E3A02001    mov   r2, #1          ; i = 1
  0x0008: E1520000    cmp   r2, r0          ; compare i with n
  0x000c: CA000002    bgt   0x1c            ; if i > n jump to 0x1c
  0x0010: E0811002    add   r1, r1, r2      ; s = s + i
  0x0014: E2822001    add   r2, r2, #1      ; i = i + 1
  0x0018: EAFFFFFA    b     0x08            ; repeat loop
  0x001c: E1A00001    mov   r0, r1          ; return s in r0
  0x0020: E12FFF1E    bx    lr              ; return to caller
```

Esempio 2: Limitazione di un valore (`clamp`)

`clamp` limita `x` all'intervallo `[lo, hi]` usando le funzioni integrate `min` e `max` , che su ARMv6 diventano mosse condizionali (`movlt` , `movgt`) senza salti.

```
-- 1. Sorgente BML (tests/bml/features.b)
function clamp(x: i32, lo: i32, hi: i32): i32
  return min(max(x, lo), hi)
end
```

```
/* 2. Equivalent C */
int32_t bmt_clamp(int32_t x, int32_t lo, int32_t hi)
{
    return bmt_min_i32(bmt_max_i32(x, lo), hi);
}
```

```
; 3. ARMv6 A32 machine code (r4 = x, r5 = lo, r6 = hi, r7/r8 = stack slots)
  0x004c: E92D4FF0    push  {r4-r10, fp, lr}
  0x0050: E1A0B00D    mov   fp, sp
  0x0054: E3A0C004    mov   ip, #4
  0x0058: E04DD00C    sub   sp, sp, ip
  0x005c: E1A04000    mov   r4, r0          ; r4 = x
  0x0060: E1A05001    mov   r5, r1          ; r5 = lo
  0x0064: E1A06002    mov   r6, r2          ; r6 = hi
  0x0068: E1A07004    mov   r7, r4          ; S[0] = x
  0x006c: E1A08005    mov   r8, r5          ; S[1] = lo
  0x0070: E1570008    cmp   r7, r8          ; compare x with lo
  0x0074: B1A07008    movlt r7, r8          ; max(x, lo): if x < lo then r7 = lo
  0x0078: E1A08006    mov   r8, r6          ; S[1] = hi
  0x007c: E1570008    cmp   r7, r8          ; compare max(x, lo) with hi
  0x0080: C1A07008    movgt r7, r8          ; min(..., hi): if r7 > hi then r7 = hi
  0x0084: E1A00007    mov   r0, r7          ; result in r0
  0x008c: E1A0D00B    mov   sp, fp
  0x0090: E8BD8FF0    pop   {r4-r10, fp, pc}
```

Esempio 3: Massimo comun divisore di Euclide (`gcd`)

`gcd` usa `abs` , un ciclo `while` e `%` . Poiché l'ARM1176 non ha un'istruzione hardware per la divisione intera, `a % b` chiama la routine A32 di 30 parole `__bml_idivmod` , che restituisce il quoziente floor in `r0` e il modulo floor in `r1` .

```
-- 1. Sorgente BML (tests/bml/features.b)
function gcd(a: i32, b: i32): i32
  a = abs(a)
  b = abs(b)
  while b ~= 0 do
    local r = a % b
    a = b
    b = r
  end
  return a
end
```

```
/* 2. Equivalent C */
int32_t bmt_gcd(int32_t a, int32_t b)
{
    a = abs(a);
    b = abs(b);
    while ((b != 0)) {
        int32_t r = bmt_imod(a, b);
        a = b;
        b = r;
    }
    return a;
}
```

```
; 3. ARMv6 A32 machine code (r4 = a, r5 = b, r6 = r)
  0x00ac: E1A07004    mov   r7, r4
  0x00b0: E3570000    cmp   r7, #0
  0x00b4: B2677000    rsblt r7, r7, #0      ; abs(a) via reverse-subtract if < 0
  0x00b8: E1A04007    mov   r4, r7
  ...
; while b ~= 0 loop body:
  0x00f0: E1A00007    mov   r0, r7          ; r0 = a
  0x00f4: E1A01008    mov   r1, r8          ; r1 = b
  0x00f8: EB0001B5    bl    __bml_idivmod   ; r0 = a // b, r1 = a % b
  0x00fc: E1A07001    mov   r7, r1          ; S[0] = remainder r1
  0x0100: E1A06007    mov   r6, r7          ; local r = a % b
  0x0104: E1A07005    mov   r7, r5
  0x0108: E1A04007    mov   r4, r7          ; a = b
  0x010c: E1A07006    mov   r7, r6
  0x0110: E1A05007    mov   r5, r7          ; b = r
  0x0114: EAFFFFEC    b     0x00cc          ; jump back to while test
```

Esempio 4: Doppia ricorsione (`fib`)

In `fib(n - 1) + fib(n - 2)` , la prima chiamata ricorsiva lascia il suo risultato in `S[0]` , che è mappato nel registro callee-saved `r5` . Quando parte la seconda `bl fn_fib` , `fn_fib` salva e ripristina `r4` .. `r10` sul proprio stack frame, così `r4` (`n`) e `r5` (`fib(n - 1)`) restano intatti senza accessi aggiuntivi in memoria.

```
-- 1. Sorgente BML (tests/bml/features.b)
function fib(n: i32): i32
  if n <= 0 then
    return 0
  elseif n == 1 then
    return 1
  else
    return fib(n - 1) + fib(n - 2)
  end
end
```

```
/* 2. Equivalent C */
int32_t bmt_fib(int32_t n)
{
    if ((n <= 0)) return 0;
    else if ((n == 1)) return 1;
    else return (bmt_fib((n - 1)) + bmt_fib((n - 2)));
}
```

```
; 3. ARMv6 A32 recursive else branch (r4 = n, r5 = S[0], r6 = S[1])
  0x0198: E1A05004    mov   r5, r4          ; S[0] = n
  0x019c: E3A06001    mov   r6, #1          ; S[1] = 1
  0x01a0: E0455006    sub   r5, r5, r6      ; S[0] = n - 1
  0x01a4: E1A00005    mov   r0, r5          ; arg0 = n - 1
  0x01a8: EBFFFFDF    bl    fn_fib          ; call fib(n - 1)
  0x01ac: E1A05000    mov   r5, r0          ; r5 (callee-saved) = fib(n - 1)
  0x01b0: E1A06004    mov   r6, r4          ; r6 = n (r4 preserved across call)
  0x01b4: E3A07002    mov   r7, #2          ; r7 = 2
  0x01b8: E0466007    sub   r6, r6, r7      ; r6 = n - 2
  0x01bc: E1A00006    mov   r0, r6          ; arg0 = n - 2
  0x01c0: EBFFFFD9    bl    fn_fib          ; call fib(n - 2) (r5 preserved)
  0x01c4: E1A06000    mov   r6, r0          ; r6 = fib(n - 2)
  0x01c8: E0855006    add   r5, r5, r6      ; r5 = fib(n - 1) + fib(n - 2)
  0x01cc: E1A00005    mov   r0, r5          ; return result in r0
```

Esempio 5: Struct, array e `f32` VFPv2 (`step_vec2f`)

La Tappa 6 traduce l'aritmetica `f32` (`+` , `-` , `*` , `/` , `sqrt` , confronti) nelle istruzioni hardware del coprocessore 10 VFPv2 (`vadd.f32` , `vmul.f32` , `vsqrt.f32` , `vcmp.f32` + `vmrs APSR_nzcv, fpscr`) e compila gli accessi ai campi delle `struct` (`v.x` , `v.y`) in letture e scritture a 32 bit.

```
-- 1. Sorgente BML (tests/bml/types_mem.b)
struct Vec2f
  x: f32
  y: f32
end

function step_vec2f(v: Vec2f, dt: f32, lim: f32): i32
  v.x = v.x + 2.0 * dt
  v.y = v.y - 1.5 * dt
  local d: f32 = sqrt(v.x * v.x + v.y * v.y)
  if d > lim then
    v.x = v.x * (lim / d)
    v.y = v.y * (lim / d)
  end
  return floor(d * 100.0)
end
```

```
/* 2. Equivalent C */
typedef struct { float x, y; } Vec2f;

int32_t step_vec2f(Vec2f *v, float dt, float lim)
{
    v->x = v->x + 2.0f * dt;
    v->y = v->y - 1.5f * dt;
    float d = sqrtf(v->x * v->x + v->y * v->y);
    if (d > lim) {
        v->x = v->x * (lim / d);
        v->y = v->y * (lim / d);
    }
    return (int32_t)floorf(d * 100.0f);
}
```

```
; 3. ARMv6 + VFPv2 coprocessor 10 machine code (distance & comparison)
  EE007A10    vmov.f32  s0, r7          ; s0 = v.x * v.x + v.y * v.y
  EEB10AC0    vsqrt.f32 s0, s0          ; hardware VFPv2 square root
  EE107A10    vmov.f32  r7, s0          ; r7 = d (IEEE-754 binary32 bits)
  EE007A10    vmov.f32  s0, r7          ; s0 = d
  EE006A90    vmov.f32  s1, r6          ; s1 = lim
  EEB40A60    vcmp.f32  s0, s1          ; compare d with lim in VFPv2
  EEF1FA10    vmrs      APSR_nzcv, fpscr; transfer VFP flags to ARM CPSR
```

Esempio 6: Ottimizzazione di una cartuccia (Dark Night)

In `carts/darknight/main.lua`, la separazione dei nemici controlla 272 coppie a ogni frame ($O(N^2)$). In Lua, i lookup nelle tabelle (`e.x` , `o.x` , `o.st`) e le operazioni float nella VM assorbono il 27,3% del tempo di calcolo. Convertire il ciclo in una funzione calda `.b` con `vm_buf` elimina il boxing delle tabelle, usa istruzioni hardware VFPv2 e riduce il tempo da 20 us a 4 us (fino a 5 volte più veloce).

```
-- 1. Originale Lua in carts/darknight/main.lua
for i = 1, #enemies do
  local o = enemies[i]
  local ox = ex - o.x
  if ox < 0.8 and ox > -0.8 and o ~= e and o.st ~= "dead" then
    local oz = ez - o.z
    local d2 = ox * ox + oz * oz
    if d2 < 0.64 and d2 > 0.000001 then
      try_move(e, ox / sqrt(d2) * 0.02, oz / sqrt(d2) * 0.02, 0.35)
    end
  end
end
```

```
-- 2. Implementazione tipata in .b (tests/bml/darknight_enemy.b)
function move_and_separate(n: i32, x: f32[], z: f32[], vx: f32[], vz: f32[], alive: bool[], out: i32[],
  for self = 1, n do
    if alive[self] then
      for i = 1, n do
        if i ~= self and alive[i] then
          local dx = x[self] - x[i]
          if dx < 0.8 and dx > -0.8 then
            local dz = z[self] - z[i]
            local d2 = dx * dx + dz * dz
            if d2 < 0.64 and d2 > 0.00001 then
              counts[self] = counts[self] + 1
              out[(self - 1) * n + counts[self]] = i
            end
          end
        end
      end
    end
  end
  return 0
end
```

```
-- 3. Integrazione zero-copy nella cartuccia con native_compile e vm_buf
local blob = assert(native_compile(b_source))
local mod = assert(native_load(blob))
local bx = vm_buf("i32", N)
local bz = vm_buf("i32", N)
native_run(mod, "move_and_separate", N, bx, bz, bvx, bvz, balive, bout, bcounts)
```

Implementazione	Scansione 17 nemici	Chiamate per frame	Costo del passaggio
Tabelle Lua originali	15 - 20 us	0 (VM Lua)	Hashing tabelle
Chiamate native multiple	8 - 12 us	34 chiamate	34 us overhead
Chiamata singola + vm_buf	3 - 4 us	1 chiamata	1 us totale

Prova passo per passo

PROVA

Compila `tests/bml/sum.b` e `tests/bml/darknight_enemy.b`, ispeziona il bytecode a stack e le parole macchina ARMv6 + VFPv2, oppure compila testo `.b` direttamente da Lua con `native_compile(src)`:

1. Esegui `python3 tools/bml/bml.py tests/bml/sum.b --dump --run sum 10`: vedi le istruzioni a stack e il risultato `55`. 2. Esegui `python3 tools/bml/bml.py tests/bml/sum.b --dump-armv6`: vedi le 9 parole esadecimali A32 da `0x0000` a `0x0020`. 3. Esegui `make test-bml`: la suite controlla 23 programmi errati, `sum.b`, `features.b`, 200 programmi casuali, `types_mem.b`, `kernel_fn.b`, `darknight_enemy.b` (60 frame di 17 nemici) e il compilatore C su console `native_compile` (Tappa 9) confrontando Lua, `StackVM`, `bmvm` in C, `native_run` e `qemu-system-arm -cpu arm1176`.

DA COPIARE

```
python3 tools/bml/bml.py tests/bml/sum.b --dump --run sum 10
python3 tools/bml/bml.py tests/bml/sum.b --dump-armv6
python3 tools/bml/bml.py tests/bml/features.b --native build/features.bmn
make test-bml
```

Punti aperti

Nessun punto aperto.